

Politecnico di Milano

Anno accademico 2012-2013

Ingegneria del Software – Appello del 17 Luglio 2013

Cognome:		
Nome:		Matricola:
Sezione (segnarne una): ☐ Baresi	☐ Ghezzi	☐ San Pietro

Istruzioni

1. La mancata indicazione dei dati anagrafici e della sezione comporta l'annullamento del compito.
2. Al termine, consegnare solo i fogli distribuiti utilizzando il retro delle pagine in caso di necessità. Non separare questi fogli. Eventuali fogli di brutta, ecc. non verranno in nessun caso presi in considerazione. È possibile scrivere in matita.
3. È possibile consultare liberamente libri, manuali o appunti. È proibito l'uso di ogni dispositivo elettronico (quali calcolatrici tascabili, telefoni cellulari, ecc.).
4. Non è possibile lasciare l'aula conservando il tema della prova in corso.
5. Tempo a disposizione: 2h.

Esercizio 1:

Esercizio 2:

Esercizio 3:

Esercizio 4:

Esercizio 1 (punti 15)

Si consideri il seguente ADT Lista-Doppia-Con-Cursore (nel seguito, abbreviata come LDCC). Una LDCC è costituita da una lista sequenziale di elementi (detti *nodi*) e da un *cursore* posizionato "a metà" fra due nodi della lista. Il cursore consente di accedere sia all'eventuale nodo alla sua sinistra che all'eventuale nodo alla sua destra, e può essere mosso sia verso sinistra che verso destra. La lista risulta quindi bidirezionale. Si considerino le seguenti dichiarazioni:

```
import java.util.*;
public class LDCC<T> {
//Overview Una LDCC e' costituita da una lista sequenziale di  nodi di tipo T,
//e da un cursore posizionato fra due nodi. Null non pu\'o essere parte della lista.
//Il cursore pu\'o essere mosso di una posizione in entrambe le direzioni, mentre un
elemento pu\'o essere aggiunto o rimosso sia alla sua sinistra che alla sua destra.

        //@ensures (*Costruisce un LDCC vuoto*)
        public LDCC() { }
//observers:
        //@ensures (*\result e' true sse a sinistra del cursore non ci sono nodi*);
        public boolean endLeft() {return true;}

        //@ensures (*\result e' true sse a destra del cursore non ci sono nodi*);
        public boolean endRight() {return true;}

        //@ensure (*\result e' l'elemento a sx del cursore*);
        public T leftItem() throws NoSuchElementException {}

        //@ensure (*\result e' l'elemento a dx del cursore*);
        public T rightItem() throws NoSuchElementException { }

        //@ensures (* \result e' costituito da tutti gli elementi di this,
        //@nell'ordine da sx a dx corrispondente a posizioni crescenti, con il cursore;
        //@segnalato da un elemento speciale, di valore null*)
        public ArrayList<T> elements() {}
//modifiers
        //muove il cursore a sx se possibile
        public void moveLeft() throws NoSuchElementException { }

        //muove il cursore a dx se possibile
        public void moveRight() throws NoSuchElementException { }

        //inserisce t a sx del cursore
        public void insertleft(T t) {}

        //inserisce t a dx del cursore
        public void insertRight(T t) {}

        //cancella, se possibile, l'elemento a sx del cursore
        public void deleteLeft() throws NoSuchElementException {  }

        //cancella, se possibile, l'elemento a dx del cursore
        public void deleteRight() throws NoSuchElementException {  }

}
```

Domanda 1

Scrivere la specifica JML del metodo insertRight, aggiungendo, se necessario, opportune eccezioni. Si consiglia di scrivere la specifica usando anche il metodo observer elements() e il metodo indexOf(Object x) di ArrayList che

restituisce la posizione a cui si trova la prima occorrenza dell'elemento x.

Soluzione: Si aggiunge NullPointerException per evitare di inserire null. Poiché l'inserimento avviene a destra, il cursore e gli elementi alla sua sinistra non cambiano posizione. A destra si inserisce l'elemento, mentrele posizioni successive sono shiftate di 1 verso dx.

```
//@ensures t!=null && elements.size()== \old(elements.size()+1) &&
//@  elements().indexOf(null) == \old(elements().indexOf(null)) &&
//@  (\forall int i; 0<= i && i <elements().size());
//@       (i<=elements().indexOf(null) ==> (elements().get(i) == \old(elements().get(i))) &&
//@       (i==1+elements().indexOf(null) ==> (elements().get(i) == t)) &&
//@       (i>1+elements().indexOf(null)) ==> (elements().get(i) == \old(elements().get(i-1)))
//signals (NullPointerException e) t==null;

public void insertRight(T t) {}
```

Domanda 2

Si consideri la seguente implementazione parziale della classe LDCC

```
public class LDCC<T> {
     private ArrayList<T> nodo;
     private int cursore;
  public LDCC() {
      nodo = new ArrayList<T>();
      cursore = -1;
  }
  public boolean endLeft() {return (cursore == -1);
  }
  public boolean endRight() {return (cursore == nodo.size()-1);
  }
  public void insertRight(T t) {
      if (t == null) throw new NullPointerException();
      nodo.add(cursore+1,t);
  }
  public void insertLeft(T t) {
      if (t == null) throw new NullPointerException();
      nodo.add(cursore+1,t);
      cursore++;
  }
  public void moveRight() throws NoSuchElementException {
      if (endRight()) throw new NoSuchElementException();
      cursore++;
  }
}
```

dove si intende che *nodo* include tutti gli elementi della lista, mentre il valore di *cursore* indica l'indice del nodo immediatamente *a sinistra* del cursore.

Scrivere l'invariante di rappresentazione della classe LDCC in JML.

```
private invariant ...
```

Soluzione:

```
private invariant
nodo!=null && cursore>=-1 && cursore < nodo().size() && !nodo.contains(null);
```

Domanda 3

Scrivere la funzione di rappresentazione della classe LDCC, usando la notazione preferita.

Soluzione: Si può usare ad esempio ancora un private invariant:

```
private invariant
elements().indexof(null) == cursore +1 &&
(\forall int i; 0<= i && i<nodo.size();
              (i<=cursore ==> (elements().get(i) == nodo.get(i))) &&
              (i>cursore ==> (elements().get(i+1) == nodo.get(i))));
```

Domanda 4

Implementare il metodo elements().

Soluzione:

```
public ArrayList<T> elements() {
        ArrayList<T> temp = (ArrayList<T>)nodo.clone();
        nodo.add(cursore+1,null);
        return temp;
        }
```

Domanda 5 (facoltativa)

Mostrare che l'implementazione di insertRight ne soddisfa la specifica.

Soluzione: Il caso $t = null$ eòvvio. Se t non e null, la add inserisce un elemento in $nodo$ alla destra della posizione cursor+1, lasciando nelle loro posizioni gli elementi precedenti e shiftando di 1 a destra quelli successivi. In base alla funzione di astrazione, si osserva che la posizione cursore +1 di nodo corrisponde alla posizione di elements() avente valore null, e che, visto che la posizione del cursore non cambia, lo stesso elemento inserito in nodo si ritrova inserito a destra della posizione null nel nuovo valore di elementi() (con gli elementi a sx immutati e a dx shiftati di 1). Ma questa eòproprio la specifica di insertRight.

Esercizio 2 (punti 6

Si consideri la classe LDCC dell'esercizio 1. La seguente classe MultiLDCC rappresenta anch'essa una LDCC, ma consente di utilizzare più cursori. La lista è associata infatti ad una pila (mai vuota) di cursori $(c_1, .., c_n)$, con $n > 0$, di cui quello in cima alla pila (c_n) è il cursore corrente.

In particolare, una lista MultiLDCC ha inizialmente un solo cursore (che è anche il corrente), ma può anche "impilare" un nuovo cursore, come copia del cursore corrente, tramite un metodo pushCursor(). Il metodo popCursor() consente di eliminare il cursore in cima alla pila. Infine, islastCursor() consente di verificare se il cursore corrente è anche l'unico sulla pila. Tutti gli altri metodi di MultiLDCC sono specificati esattamente come quelli di LDCC, considerando che ciascuo di questi metodi si riferisce implicitamente al cursore corrente. Ad es. elements() restituisce sempre un ArrayList che contiene tutti e soli gli elementi della lista, con un elemento=null nella posizione del cursore corrente; moveLeft() muove a sinistra di una posizione il cursore corrente, ecc.

Domanda 1

È possibile definire MultiLDCC come erede di LDCC, rispettando il principio di sostituzione? E' necessario motivare accuratamente la risposta.

Soluzione: Poiché la specifica dei metodi ereditati non cambia, sia la regola della segnatura che la regola dei metodi sono verificate. Occorre quindi stabilire se vale la regola delle proprietà. Nessuna delle proprietà elencate nella Overview della classe cambia avendo a disposizione più cursori (visto che se ne sua sempre uno solo alla volta), quindi il principio risulta rispettato. Il tutto dipende dall'ipotesi (del testo) che la pila dei cursori non sia mai vuota, e quindi la lista abbia senmpre un cursore corrente.

Domanda 2

Come fareste a specificare in JML il metodo pushCursor, supponendo di avere a disposizione il tipo di dato astratto Pila<T>? Si supponga che la specifica di Pila sia quella tradizionale, con push, pop, top, size(), ecc., ma con un metodo osservatore content() che restituisce in un ArrayList tutti gli elementi della pila, dal fondo alla cima. Si noti che le informazioni disponibili non sono complete poter specificare il metodo pushCursor: è necessario aggiungere ulteriori elementi alla specifica.

Soluzione: Non vi è alcun tipo di dati corrispondente ai cursori, mentre per specificare il comportamento a pila occorre conoscere il tipo degli elementi (es. Pila<Integer>, ecc.).. Una soluzione è quella di aggiungere un metodo osservatore pilaCursori() che restituisce una Pila, in cui ogni elemento della Pila non è un cursore, ma una LDCC che contiene gli stessi elementi della MultiLDCC (che si suppone essere definita come erede di LDCC), ma ciascuno con il suo cursore.

```
//@ensures (* tutti gli elementi() contenuti in \result sono identici a this.elementi(),
//@          a parte il cursore.*);
public Pila<LDCC<T>>  pilaCursori()
```

E' a questo punto facile definire:

```
//@ensures pilaCursori().top().equals(this) &&
//@  pilaCursori().size() == \old(pilaCursori().size()+1) &&
//@  (\forall int i; 0\le i && i<content().size()-1; content().get(i)==\old(content().get(i)));
public void pushCursor()
```

Esercizio 3 (punti 5)

Il seguente metodo Java prende in ingresso un array di al più 5 interi e restituisce l'elemento più grande in valore assoluto, oppure 0 se l'array fosse vuoto oppure -1 se troppo grande.

```
public int max(int[] numbers) {
   int i, max_value = 0;
   if (numbers.length > 5) return -1;
   for (i = 0; i<numbers.length; i++) {
      if (numbers[i] < 0)
         max_value = Math.max(max_value,Math.abs(numbers[i]));
      else max_value = Math.max(max_value, numbers[i]);
   }
   return max_value;
}
```

- Disegnare il diagramma del flusso di controllo.

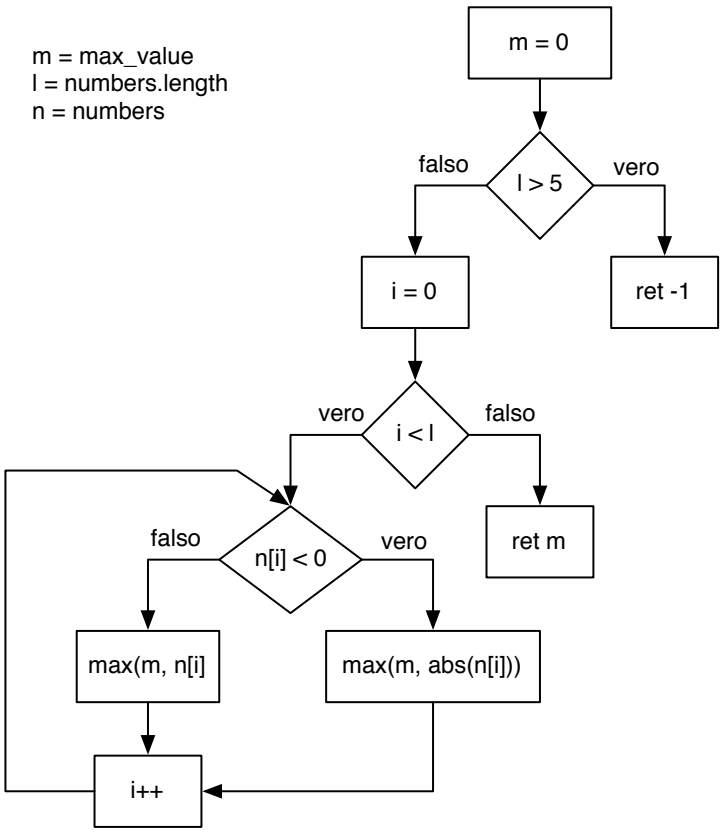
  Soluzione:

- Il metodo è collaudato con i casi di test seguenti (input; valore restituito):
  - T1: {0,0,0,0,0}; 0;
  - T2: {1,2,3,4,5}; 5;
  - T3: {-1,-2,-3,-4,-5}; 5;
  - T4: {1,2,3,4,5,6}; -1;
  - T5: {-10,10,3,5,-6}; 10;
  - T6: {}; -1;

  1. Definire le percentuali di copertura delle istruzioni (*statement*) e delle decisioni (*branch*) ottenute eseguendo ogni test separatamente.
     Soluzione:
     Come misura del numero dei branch consideriamo tutti gli archi del diagramma di flusso di controllo.

T	statement (10)	branch/archi (12)
T1	8	8
T2	8	8
T3	8	8
T4	3	2
T5	9	10
T6	5	4

  2. Quale test darebbe errore? **Soluzione:** T6: restituisce 0 e non -1;



Esercizio 4 (punti 7)

Diversi corridori partecipano a una corsa ciclistica a tappe. Ogni corridore appartiene a una squadra e ogni squadra deve schierare 9 corridori al via della manifestazione. Ogni tappa può essere una prova in linea o a cronometro. Ogni corridore può iniziare una tappa e ritirarsi durante il suo svolgimento, o alla fine della medesima. Se una squadra resta con meno di 4 corridori, l'intera squadra è costretta al ritiro. Se completa il percorso, ogni corridore acquisisce un tempo di tappa che verrà utilizzato per stilare la classifica. Questa ovviamente deve elencare solamente i concorrenti ancora in gara, ovvero coloro che non si sono ritirati. La classifica deve tener conto degli abbuoni assegnati al termine di ogni tappa: 20' al primo classificato, 15' al secondo e 10' al terzo.

- Definire il diagramma delle classi per realizzare il sistema di gestione della corsa. È richiesta anche la definizione dei metodi e degli attributi principali delle classi identificate.
- Realizzare un diagramma di sequenza per spiegare come funziona il metodo che calcola la classifica generale alla fine di ogni tappa.
- Scrivere l'invariante della classe `Squadra` per imporre il numero minimo di atleti in gara.

Soluzione: